

Damage-Aware KL Anchoring for Small Language Models

From AI and LLM fundamentals to the complete experiment, evaluation, team plan, and novelty claim

0.5B-1.5B	10	8 weeks
<i>recommended model scale</i>	<i>central training runs</i>	<i>course-project plan</i>

Core idea: use a short pilot fine-tuning run to find the general prompts that are most vulnerable, then spend a small fixed KL-protection budget on those prompts during the real LoRA training run.

Prepared as a self-contained learning resource

Project framing: research-oriented LLM course project for a 3-4 person team

The complete project in one picture

The pilot diagnoses vulnerability; it is then discarded. Every final condition restarts from the same base model.



A beginner-first study guide to the complete research project

This guide explains the proposed course project from the beginning. It assumes no prior knowledge of artificial intelligence, machine learning, language models, fine-tuning, LoRA, knowledge distillation, KL divergence, or catastrophic forgetting.

1. The project in one paragraph

We begin with a small language model that has broad general abilities. We fine-tune it with LoRA so that it becomes better at a narrow target task, such as mathematical word problems. Fine-tuning may unintentionally make the model worse at unrelated abilities. To reduce this damage, we occasionally show the fine-tuned model general prompts and encourage it to behave like its original frozen version. The ordinary approach chooses these protective prompts randomly. Our proposed research asks whether a short pilot fine-tuning run can identify which prompts are most vulnerable, so that a small fixed protection budget can be spent on the prompts that need it most.

One-sentence research question: Under the same target-training, anchor-token, and compute budget, does pilot-guided selection of vulnerable prompts preserve general capabilities better than random or diversity-based KL anchoring?

2. What artificial intelligence means

Artificial intelligence, or AI, is a broad name for computer systems that perform tasks associated with intelligence. Examples include recognizing objects in photographs, translating languages, recommending films, answering questions, and writing code.

AI does not need to think like a person. A useful way to understand modern AI is that it learns very complicated statistical patterns from examples and then uses those patterns to produce predictions.

3. What machine learning means

In traditional programming, a programmer writes the rules directly. A simple program might say: if a student's mark is at least 50, return "Pass"; otherwise, return "Fail."

In machine learning, we give the computer examples and allow it to learn the rule. For sentiment classification, it might see "I loved this movie" labelled positive and "The film was terrible" labelled negative. After enough examples, the model learns patterns that help it classify new sentences.

4. What a model is

A model is a large mathematical function containing adjustable values called parameters. You can imagine the parameters as millions or billions of small knobs. Changing them changes how the model transforms an input into an output.

For a language model, the flow is simple to state: a user provides text, the model processes it, and the model predicts what text should come next.

5. What a neural network is

A neural network is a model built from many layers of mathematical operations. Early transformations detect useful local patterns; later transformations combine them into more complicated representations. In a real language model, knowledge and behaviours are distributed across many layers and parameters rather than stored in neat, human-readable boxes.

6. What an LLM is

LLM means large language model. An LLM learns from an enormous quantity of text. Its central pretraining exercise is usually next-token prediction.

Given the text "The capital of France is", the model assigns a probability to every possible next token. It might assign 90% to Paris, 4% to London, 2% to Berlin, and the remaining probability to other possibilities. The complete set of probabilities is called a probability distribution.

The model then chooses or samples a token, appends it to the text, and predicts the next token again. Repeating this process produces a response.

7. Pretraining and the base model

Pretraining is the large initial learning process that creates the base model. The model reads books, websites, articles, code, and many other texts. It learns language structure, common facts, reasoning patterns, writing styles, and relationships between concepts.

The resulting base model is like a student who has received a broad education. It may know a little about mathematics, science, history, writing, programming, and many other areas, but it has not necessarily been optimized for one narrow application.

8. Fine-tuning

Fine-tuning takes a pretrained model and trains it further on a smaller specialized dataset. Suppose we want the model to solve mathematical word problems. We can fine-tune it on examples from GSM8K.

An example question might say that Ravi has 12 apples and gives 5 away. The desired response explains that 12 minus 5 equals 7. Repeated examples teach the model the style and reasoning expected by the target task.

The target-task loss is a number representing how different the model's predictions are from the desired responses. Training adjusts parameters to reduce this loss.

9. Catastrophic forgetting

Imagine a student named Maya who has studied history, science, mathematics, literature, and geography. Maya then attends an intensive course that rewards only mathematical answers. Afterward, she may become better at mathematics but begin answering unrelated questions in a mathematical style or perform worse in subjects she stopped practising.

An LLM can behave similarly. A hypothetical model might improve from 30% to 60% on mathematics while dropping from 50% to 43% on general knowledge and from 70% to 61% on instruction following.

This degradation is called catastrophic forgetting or capability degradation. It does not always mean that information has been completely erased. Sometimes the model still contains useful information but no longer accesses or expresses it correctly.

10. Full fine-tuning and LoRA

Full fine-tuning permits all or most of the original model parameters to change. This provides considerable learning capacity but requires substantial memory and can disturb many behaviours.

LoRA means Low-Rank Adaptation. It freezes the original parameters and adds small trainable matrices to selected layers. Instead of rewriting a giant textbook, LoRA is like attaching a small specialized notebook to it.

For an original weight matrix W_0 , ordinary LoRA uses an effective weight:

$$W' = W_0 + \Delta W$$

The update ΔW is represented by two small matrices whose product has rank r . Because r is small, far fewer values are trained than in full fine-tuning.

The frozen backbone does not guarantee preserved behaviour. The active LoRA update participates in every prediction and can still change outputs on unrelated prompts.

11. The teacher-student idea

To protect general behaviour, keep an unchanged copy of the base model. This frozen copy is the teacher. The model being fine-tuned with LoRA is the student.

The teacher represents the behaviour we want to retain. It is never updated. The student must learn the target task while remaining reasonably close to the teacher on selected general prompts.

12. Anchor prompts

An anchor prompt is a general prompt used to remind the student how the original model behaved. Examples include "What is photosynthesis?", "Write a polite email", "What is the capital of Japan?", and "Choose the logically valid conclusion."

Think of anchor prompts as revision cards given to Maya during her mathematics course. They periodically remind her that she must preserve abilities outside mathematics.

13. Knowledge distillation

Knowledge distillation usually means training a student model to imitate information produced by a teacher model. The teacher can provide more than a single correct answer. It provides a full probability distribution over possible tokens.

For "The capital of France is", the teacher might assign 90% to Paris, 4% to London, 2% to Berlin, 1% to Rome, and 3% elsewhere. After mathematics fine-tuning, the student might assign only 52% to Paris and distribute much more probability across the alternatives.

Both models still choose Paris, but the student's internal output behaviour has drifted. Comparing only the final word would miss this change. Comparing probability distributions detects it.

14. KL divergence

KL divergence is a measurement of the difference between two probability distributions. Low KL divergence means the teacher and student behave similarly. High KL divergence means their predictions differ substantially.

The formula is:

$$KL(P \parallel Q) = \text{sum over tokens of } P(\text{token}) \times \log(P(\text{token}) / Q(\text{token}))$$

Here P is the frozen teacher distribution and Q is the student distribution. The formula does not need to be memorized. It is simply a difference meter for probabilistic behaviour.

The total training loss becomes:

$$\text{Total loss} = \text{Target-task loss} + \lambda \times \text{Anchor KL loss}$$

Lambda is a strength control. A small lambda allows aggressive target learning but gives weak protection. A large lambda strongly preserves the original behaviour but may make the target task difficult to learn.

15. Why ordinary random anchoring may waste budget

Suppose 2,000 general prompts are available, but the project can afford to use only 128 during training. Random selection might choose many redundant or undamaged prompts, such as several different greeting requests, while missing capabilities that the target fine-tuning actually harms.

The central idea is to spend the limited anchor budget on prompts that appear most vulnerable to the specific fine-tuning task.

16. Damage-aware anchor selection

Step 1: establish the base reference

Evaluate the untouched model on the target task and on a small suite of unrelated capabilities. Record its probability distributions on the candidate anchor prompts.

Step 2: run a short pilot fine-tuning experiment

Create a temporary copy of the base model and perform a short LoRA warm-up on a small portion of the target data. This pilot is not one of the final models. It is a diagnostic probe that shows which behaviours begin to change.

Step 3: score candidate prompts

For each candidate prompt, compare the frozen base model with the pilot model. A simple damage score is the average token-level KL divergence between their output distributions. A high score means that the pilot fine-tuning changed behaviour strongly on that prompt.

For example, a greeting prompt might receive a damage score of 0.02, a science explanation 0.31, and a logical reasoning prompt 0.74. The reasoning prompt appears most vulnerable.

Step 4: create alternative anchor sets

Random selection chooses prompts without using damage information. Diversity selection chooses prompts from different semantic clusters. Damage selection chooses the highest-scoring prompts. Diverse-damage selection first finds damaged prompts and then removes redundancy so that several capability regions are represented.

Step 5: discard the pilot and reset

The pilot model is discarded. Every main condition starts from the same untouched base model. This prevents the selection run from giving one final condition extra target training.

Step 6: perform the main LoRA training

Training alternates between target-task batches and anchor batches. Target batches teach the new task. Anchor batches encourage the student to match the teacher's distributions.

Step 7: evaluate learning, retention, and recovery

Measure target-task improvement, general-capability change, individual correct-to-incorrect transitions, format failures, prompt robustness, and recovery under alternative evaluation methods.

17. Stable teacher outputs for unlabeled prompts

An unlabeled prompt may not include a response. One practical solution is to let the frozen base model generate a short response once, save it, and run both teacher and student over the same prompt-response sequence. KL divergence is computed at the response-token positions.

Teacher distributions can be computed during training or cached in advance. Computing them live is simple conceptually but roughly adds a teacher forward pass. Caching reduces repeated computation but requires storage. For a student project, cached responses and possibly cached top-token probabilities are practical.

18. Experimental conditions

Condition	Training behaviour	Scientific role
Base model	No fine-tuning	Starting reference
Task-only LoRA	Target examples only	Ordinary forgetting baseline
Random-KL	Random general anchors	Plain anchoring baseline
Diversity-KL	Semantically broad anchors	Coverage baseline
Damage-KL	Highest pilot-damage anchors	Proposed selector
Diverse-Damage-KL	Damaged anchors with redundancy control	Proposed strongest selector
Random labelled replay	Old examples with labels, if included	Strong practical replay baseline

The central comparison is Random-KL versus Damage-KL versus Diverse-Damage-KL. They must receive approximately equal target examples, anchor tokens, optimizer steps, and teacher computation.

19. KL anchoring versus replay

Labelled replay stores an old prompt and its correct answer, then trains the model on that answer again. KL anchoring can use unlabeled prompts and asks the student to preserve the teacher's complete probability distribution.

Replay means: remember this correct answer. KL anchoring means: behave similarly to your original self on this input.

20. Evaluation metrics

Target-task learning

For GSM8K, use numerical exact-match accuracy and target loss. Exact match checks whether the extracted final number is correct. Target loss supplies a smoother training signal.

General-capability retention

Evaluate the model before and after fine-tuning on several unrelated capabilities. Forgetting for a benchmark can be reported as the base score minus the fine-tuned score.

Item-level transitions

Classify each evaluation item as retained, forgotten, newly learned, or still incorrect. Correct before and incorrect after is a forgotten item. Incorrect before and correct after is positive backward transfer.

Anchor efficiency

Report retention improvement per fixed quantity of anchor tokens or teacher computation. This prevents a method from appearing better merely because it received more protection data.

Format and instruction compliance

Count parser failures, refusals, wrong answer schemas, and unnecessary over-generation. A benchmark drop can come from output format rather than knowledge loss.

21. Recoverable versus durable behavioural loss

An incorrect answer after fine-tuning does not prove that the underlying knowledge was erased. The model might misunderstand the prompt format or fail to express accessible information.

Use a recovery battery. First evaluate the standard prompt. Then try several meaning-preserving prompt templates. For multiple-choice questions, score answer likelihoods directly rather than relying only on generated letters. Next, provide a few demonstrations. Optionally, test a small format-only repair set.

Report categories such as prompt-recoverable, few-shot-recoverable, intervention-recoverable, and residual loss. Call the last category residual or durable behavioural loss, not proven knowledge erasure.

22. Target matching

A method can appear to forget less simply because it learned less. Suppose task-only LoRA reaches 65% mathematics accuracy and forgets eight points, while Damage-KL reaches only 42% and forgets two points. The comparison does not establish superior retention because target learning is very different.

A fair comparison selects checkpoints with approximately equal target performance, perhaps within one or two percentage points. It can also plot the complete target-learning versus forgetting frontier.

23. Falsifiable hypotheses

- Damage-selected anchors will preserve more general capability than random anchors under the same anchor-token budget.
- Diverse-damage selection will outperform simple top-damage selection when the most damaged prompts are redundant.
- Retention improvements will appear in likelihood and multi-template evaluation, not only generated exact match.
- Some apparent anchoring gains will disappear after recovery controls, revealing preserved alignment rather than additional durable capability.
- Excessively strong KL regularization will improve retention while reducing target-task learning.

24. How to interpret possible results

If damage-aware selection wins at matched target performance, the result supports targeted protection of vulnerable behaviour. If random and damage selection tie, general anchors may be highly redundant or the pilot score may be noisy. If diversity wins, broad coverage matters more than vulnerability. If damage selection loses, it may concentrate on unstable, overly difficult, or unrepresentative prompts. If raw accuracy improves but recovery-adjusted accuracy does not, the method probably protects prompt alignment or formatting rather than deeper capability.

A well-powered null result is still useful. The project should report confidence intervals, individual seed values, and failure analysis rather than hiding an inconclusive result.

25. What is and is not novel

Plain LoRA plus KL regularization is not novel. Knowledge distillation, anchoring, replay, and self-distillation already exist, and the team's original discussion included a KL regularizer.

The potential contribution is the controlled combination of pilot-estimated damage, selection of unlabeled anchors, a fixed anchor and compute budget, comparison with random and diversity selection, target-matched checkpoints, and recovery-aware measurement.

The novelty claim should be modest: the project studies whether pilot-estimated behavioural damage is useful for selecting a small unlabeled KL anchor set. Novelty confidence is medium and should be checked again with a focused search immediately before proposal submission.

26. Suggested model and data

Use one open 0.5B model for the initial complete pipeline. A 1B-1.5B model can provide a confirmation run if hardware permits. GSM8K is a practical target because it has public training and test data and a clear numerical metric.

Use approximately 2,000 filtered general instruction prompts as the anchor candidate pool. Do not use final benchmark questions as anchors. Retention evaluation can use small disjoint subsets of ARC-Challenge, BoolQ, OpenBookQA, HellaSwag, unrelated MMLU subjects, and a compact instruction-following set.

27. Realistic run budget

Five central conditions with two seeds require ten main runs: task-only LoRA, Random-KL, Diversity-KL, Damage-KL, and Diverse-Damage-KL. Only after these work should the team add anchor-budget sweeps, a second model, labelled replay, or a second target task.

28. Team division

Team member	Main responsibility
Member A	Target data, retention datasets, contamination checks, evaluation
Member B	LoRA training, KL loss, teacher caching, checkpoints
Member C	Candidate pool, pilot damage scores, diversity and damage selectors
Member D	Recovery evaluation, statistics, experiment tracking, plots and report integration

Every member should reproduce at least one component owned by another member so that the final result does not depend on one person's machine or undocumented code.

29. Eight-week plan

Week	Milestone
1	Select model and data; reproduce base evaluation; freeze splits and prompts
2	Establish task-only LoRA; confirm target learning and measurable forgetting
3	Implement and validate ordinary Random-KL anchoring
4	Run pilot; compute damage; implement diversity and damage selection
5	Run the five-condition central matrix and save checkpoints
6	Repeat central conditions with the second seed; run one essential ablation
7	Recovery evaluation, confidence intervals, and qualitative error analysis
8	Final plots, report, presentation, reproducibility check, and suspicious reruns

30. Important mistakes to avoid

- Do not use final evaluation questions as training anchors.
- Keep target data, anchor candidates, recovery examples, validation data, and final tests disjoint.
- Give selection methods equal anchor tokens and teacher computation.
- Do not compare only the final epoch; save checkpoints and target-match them.
- Do not report only average accuracy; include item transitions and format failures.
- Use at least two seeds for central comparisons.
- Do not say a method preserved knowledge if it simply failed to learn the target task.
- Do not call residual behavioural loss proven erasure.
- Record model version, dataset version, prompt templates, random seeds, hardware, wall time, memory, and every hyperparameter.

31. Final story

A broadly capable language model is sent to an intensive mathematics course. The course improves mathematics but may disturb other abilities. We keep an unchanged copy of the original model as a teacher. During the main course, the student occasionally receives general revision cards and is asked to behave like the teacher on them.

Because only a small number of cards can be used, a short pilot course first discovers which general prompts are most affected. The real course then protects those vulnerable prompts. We compare this strategy with random and diversity-based cards using equal resources. Finally, we test whether apparent forgetting is true durable behavioural loss or a recoverable problem with prompting and answer format.

Final project pitch: We investigate whether a short pilot fine-tuning run can identify general prompts that are most vulnerable to capability degradation, and whether selectively preserving the base model's behaviour on those prompts provides better retention than random KL anchoring under the same data, target-learning, and compute budget.